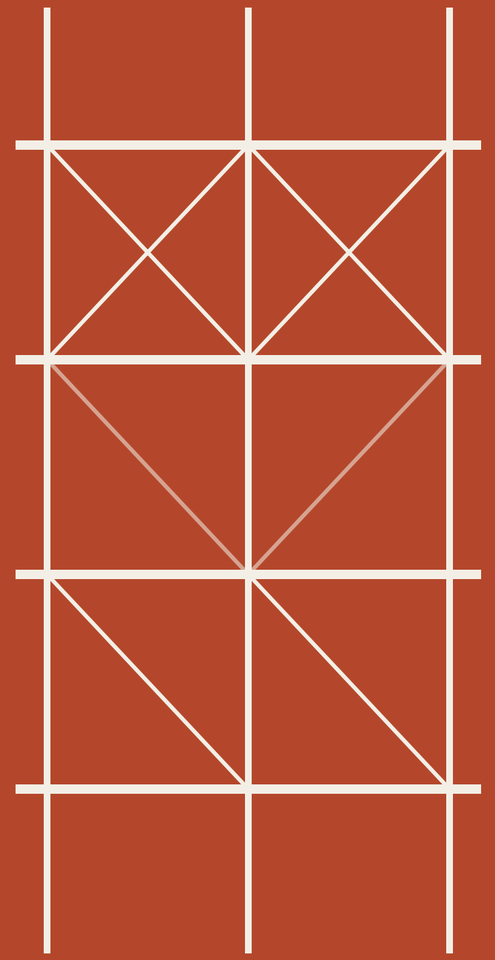
МАРКЕТИНГ ПРОГРАММНЫХ ПРОДУКТОВ И УСЛУГ

Что такое *harness?*

Разбор на примере Claude Code, OpenAI и LangChain

Мешков Владислав · группа 477901 · БГУИР, 2026

По материалам статьи Akshay, перевод на Хабре



Леса сами ничего не строят, но без них до верхних этажей не добраться

Демо работает. *Production* — нет.

раз

Модель забывает, что делала три шага назад

два

Вызовы инструментов падают без ошибок

три

Контекстное окно забивается мусором

Проблема не в модели.
Проблема *во всём, что её окружает.*

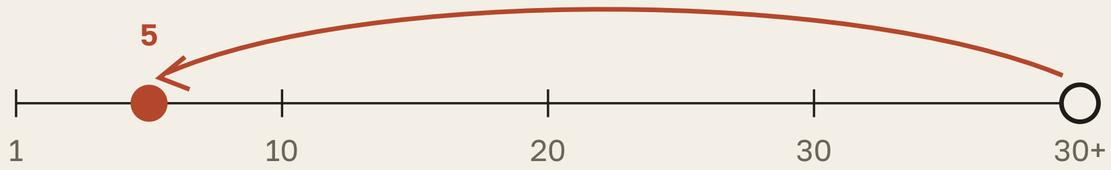
LANGCHAIN

Та же модель, те же веса — *другая обвязка*

TERMINALBENCH 2.0 · МЕСТО В РЕЙТИНГЕ

30+ → *5*

Агент поднялся из-за пределов топ-30 на 5-е место, изменив только инфраструктуру вокруг LLM.



ОТДЕЛЬНОЕ ИССЛЕДОВАНИЕ

76,4%

pass rate: LLM оптимизировал саму инфраструктуру и превзошёл системы, спроектированные вручную.

У этой инфраструктуры теперь есть название — **agent harness**.

Источник: статья Akshay (перевод на Хабре)

AGENT HARNESS

Всё, что не модель

«Если ты не модель — ты *харнесс*»

Vivek Trivedy, LangChain

ЧТО ЭТО

Вся программная инфраструктура вокруг LLM: цикл, инструменты, память, контекст, состояние, ошибки, ограничения.

АГЕНТ И HARNESS

Агент — результирующее поведение, с которым общается пользователь. **Harness** — механизм, который это поведение производит.

НА ПРАКТИКЕ

Кто говорит «я построил агента» — построил harness и направил его на модель.

BEREN MILLIDGE, 2023

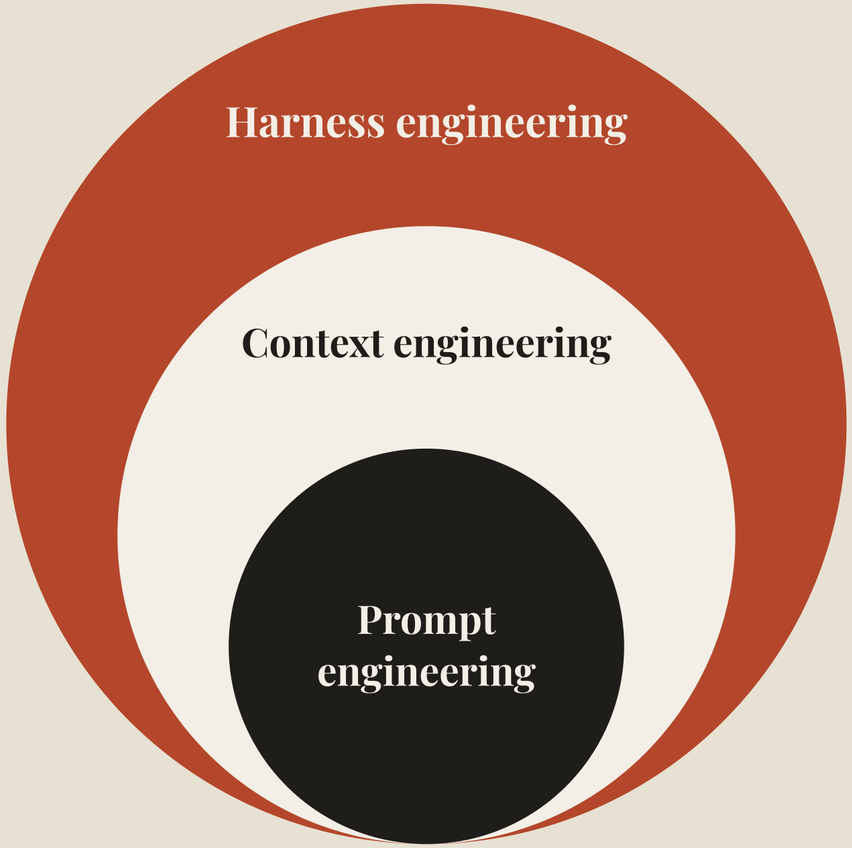
Harness — это *операционная система*

В КОМПЬЮТЕРЕ		В LLM-СИСТЕМЕ
CPU без RAM, диска и I/O	→	**Голая LLM**
RAM быстрая, но ограниченная	→	**Контекстное окно**
Диск большой, но медленный	→	**Внешние базы данных**
Драйверы устройств	→	**Интеграции с инструментами**
Операционная система	→	**Harness**

«Мы заново изобрели архитектуру фон Неймана»

ТРИ КОНЦЕНТРИЧЕСКИХ УРОВНЯ

Вокруг модели *три слоя*



Prompt engineering

Формирует инструкции, которые получает модель.

Context engineering

Управляет тем, что модель видит и когда.

Harness engineering

Оба предыдущих плюс вся прикладная инфраструктура: оркестрация инструментов, состояние, восстановление после ошибок, проверка, безопасность, жизненный цикл.

PRODUCTION-ХАРНЕСС

Из чего он *состоит*

1 **Оркестрационный цикл**
сердцебиение системы

2 **Инструменты**
руки агента

3 **Память**
внутри сессии и между сессиями

4 **Управление контекстом**
борьба с context rot

5 **Конструирование промпта**
что модель видит на каждом шаге

6 **Парсинг вывода**
структурированные tool_calls вместо текста

7 **Управление состоянием**
чекпоинты и возобновление работы

8 **Обработка ошибок**
сбои не должны накапливаться

9 **Ограничители и безопасность**
что агенту разрешено

10 **Циклы верификации**
отличие демо от production

11 **Оркестрация субагентов**
делегирование подзадач

В статье заявлено 12 компонентов, в тексте подробно описаны 11

THOUGHT → ACTION → OBSERVATION

Сердцебиение агента — *обычный цикл*

```
пока задача не решена:
    промпт = собрать(контекст)
    ответ  = llm(промпт)
    если вызовов нет:
        вернуть ответ      # финал
    результаты = выполнить(вызовы)
    контекст  += результаты
```

Схема, не реальный код. Anthropic называет свой runtime «тупым циклом»: интеллект живёт в модели.

КОГДА ЦИКЛ ЗАКАНЧИВАЕТСЯ

ответ без вызовов, лимит ходов, бюджет токенов, tripwire, прерывание пользователем, safety refusal

1. **Сборка промпта**
   всё, что увидит модель
2. **Вызов модели**
   текст и/или запросы на вызов инструментов
3. **Разбор ответа**
   вызовов нет — конец, есть — идём дальше
4. **Выполнение инструментов**
   права, изоляция, параллельное чтение
5. **Упаковка результатов**
   ошибка — тоже результат: модель исправится
6. **Обновление контекста**
   при нехватке места запускается compaction
7. **Повтор**
   пока не сработает условие завершения

Простой вопрос — 1–2 итерации, сложный рефакторинг — десятки вызовов

РУКИ АГЕНТА

Инструменты *и разбор вывода*

Как это устроено

Инструмент описан схемой: имя, описание, типы параметров. Схема попадает в контекст, и модель знает, что ей доступно.

- регистрация и проверка схем
- извлечение аргументов
- изолированное выполнение
- результат — в виде наблюдения

Claude Code

Шесть категорий инструментов:

- файловые операции
- поиск
- выполнение
- веб-доступ
- анализ кода
- запуск субагентов

Agents SDK OpenAI

Три вида инструментов:

- `@function_tool` — свои функции
- hosted tools: WebSearch, CodeInterpreter, FileSearch
- инструменты MCP

Парсинг вывода

Современные harness’ы опираются на native tool calling: модель возвращает структурированные **tool_calls**, а не свободный текст. Вызовы есть — выполняем и продолжаем цикл; нет — это финальный ответ.

MEMORY

Память работает *в нескольких масштабах*

ДВА ГОРИЗОНТА

Краткосрочная

история диалога в рамках одной сессии

Долгосрочная

сохраняется между сессиями: файлы CLAUDE.md и MEMORY.md (Anthropic), JSON-хранилища с namespace (LangGraph), Sessions на SQLite или Redis (OpenAI)

ТРИ УРОВНЯ В CLAUDE CODE

ВСЕГДА ЗАГРУЖЕН

Лёгкий индекс

около 150 символов на запись

ПО ТРЕБОВАНИЮ

Файлы по темам

подгружаются, когда нужны

ТОЛЬКО ЧЕРЕЗ ПОИСК

Сырые транскрипты

полная история работы

Агент считает свою память подсказкой и сверяет её с актуальным состоянием, прежде чем действовать.

CONTEXT ROT

Контекст — *дефицитный ресурс*

30%+

на столько падает качество, когда важное оказывается в середине окна.

Исследование Chroma, подтверждённое «Lost in the Middle» (Stanford). Даже окна в миллион токенов теряют в следовании инструкциям. Поэтому важное кладут в начало и конец промпта.

1 **Compaction**
суммаризация истории у лимита; Claude Code сохраняет архитектурные решения и нерешённые баги

2 **Observation masking**
старые выводы инструментов скрыты, вызовы видны (JetBrains Junie)

3 **Just-in-time retrieval**
лёгкие идентификаторы, данные грузятся по ходу: grep, glob, head, tail вместо целых файлов

4 **Делегирование субагентам**
исследуют широко, возвращают саммари на 1000–2000 токенов

Цель — минимальный набор высокосигнальных токенов, который максимизирует вероятность нужного результата.

ERROR HANDLING

Ошибки *накапливаются*

$0{,}99^{10} \approx 0{,}904$

Процесс из 10 шагов с 99% успеха на каждом даёт лишь около **90,4%** сквозного успеха.

2 — столько retry разрешает production-harness Stripe.

ЧЕТЫРЕ ТИПА ОШИБОК В LANGGRAPH

Транзиентные	повторить с задержкой **(retry with backoff)**
Исправимые моделью	вернуть ошибку как **ToolMessage**, пусть скорректируется
Исправимые человеком	прервать работу и запросить ввод
Неожиданные	пробросить наружу для отладки

Anthropic перехватывает сбои в обработчиках инструментов и возвращает их как результат, чтобы цикл продолжал работу

GUARDRAILS

Модель решает, что попытаться. *Система — что разрешено*

Input guardrails

запускаются на первом агенте

Tool guardrails

запускаются при каждом вызове инструмента

Output guardrails

запускаются на финальном выводе

Три уровня в SDK OpenAI. Механизм tripwire немедленно останавливает агента.

Anthropic: разрешения отделены от рассуждений модели

Claude Code независимо контролирует около 40 возможностей инструментов.

Три стадии

Доверие при загрузке проекта → проверка перед каждым вызовом → явное подтверждение пользователя для рискованных операций.

VERIFICATION LOOPS

Проверка отделяет *демо от production*

i

Правила

Тесты, линтеры, проверка типов — детерминированная истина.

ii

Картинка

Скриншоты через Playwright для задач с интерфейсом.

iii

LLM как судья

Отдельный субагент оценивает результат: ловит смысловые проблемы, но добавляет задержку.

2–3×

Подходы, которые рекомендует Anthropic

Boris Cherny, создатель Claude Code: способность проверять свою работу улучшает качество модели в 2–3 раза.

Thoughtworks (команда Мартина Фаулера): **guides** направляют до действия, **sensors** наблюдают после.

SUBAGENT ORCHESTRATION

Как делегируют *подзадачи*

Claude Code	**Fork** — байт-в-байт копия родительского контекста. **Teammate** — отдельная панель терминала, общение через файлы-mailbox. **Worktree** — отдельный git worktree: своя ветка на каждого агента.
Agents SDK OpenAI	**Agents-as-tools** — специалист решает ограниченную подзадачу. **Handoffs** — специалист берёт полное управление.
LangGraph	Субагенты — вложенные графы состояний.

> **Сначала один агент.** Anthropic и OpenAI советуют максимально использовать одного. Делить стоит, когда пересекающихся инструментов больше десяти или домены задач явно разделены: у мультиагентности есть накладные расходы на маршрутизацию и потери контекста при handoff.

ОДИН ПАТТЕРН, РАЗНЫЕ ПОДХОДЫ

Как harness строят *на практике*

ФРЕЙМВОРК	КАК УСТРОЕН	ЧТО ОСОБЕННОГО
Claude Agent SDK	Одна функция `query()` возвращает async-итератор сообщений	Цикл Gather → Act → Verify; интеллект в модели
OpenAI Agents SDK	Класс Runner: async, sync, streamed	Code-first: логика на Python, без графовых DSL. Codex: Core, App Server, клиенты с одним harness'ом
LangGraph	Граф состояний: узлы `llm_call` и `tool_node`	Условное ребро: есть вызовы — в tool_node, нет — в END. Вырос из AgentExecutor
CrewAI	Agent, Task, Crew: роли, цели, предыстория	Flows — детерминированный скелет вокруг автономных Crews
AutoGen	Core, AgentChat, Extensions	Пять паттернов: sequential, concurrent, group chat, handoff, magentic

AutoGen эволюционирует в Microsoft Agent Framework

RALPH LOOP · ANTHROPIC

Когда задача длиннее *одного контекстного окна*

ОДИН РАЗ

Initializer Agent

— init-скрипт
— progress-файл
— список фич
— первый git-коммит

→

КАЖДАЯ СЛЕДУЮЩАЯ СЕССИЯ

Coding Agent

1. Читает git-логи и progress-файл, чтобы сориентироваться
2. Берёт незавершённую фичу с наивысшим приоритетом
3. Работает над ней
4. Делает коммит и пишет саммари

Файловая система обеспечивает преемственность между окнами: git-коммиты — чекпоинты, progress-файлы — черновики.

LANGGRAPH

Состояние — типизированные словари; чекпоинты на границах super-step: возобновление и отладка «с машиной времени».

OPENAI

Четыре стратегии: память приложения, SDK sessions, Conversations API на сервере или цепочка **previous_response_id**.

СТРОИТЕЛЬНЫЕ ЛЕСА

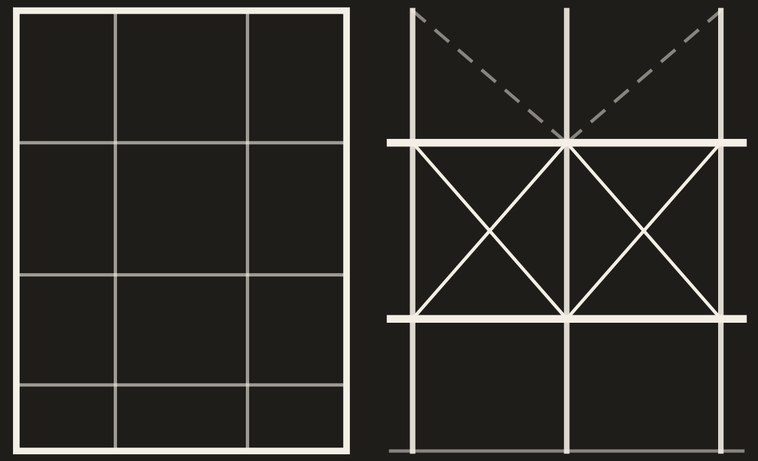
Леса убирают, *когда здание готово*

По мере улучшения моделей сложность харнесса должна *снижаться*.

Manus перестраивали пять раз за шесть месяцев, каждый раз убирая сложность: хитрые определения инструментов стали общим shell, «управляющие агенты» — простыми handoff’ами.

Ко-эволюция: модели дообучают со своими харнессами. Смена реализации инструментов может ухудшить результат.

> Тест на будущее: качество растёт с более мощной моделью, а сложность харнесса — нет. Значит, дизайн верный.



СЕМЬ РЕШЕНИЙ

Что выбирает *каждый архитектор*

1	**Один агент или несколько**	Сначала один. Делить, когда больше ~10 пересекающихся инструментов.
2	**ReAct или plan-and-execute**	ReAct гибок, но дороже на шаг. LLMCompiler: ускорение **в 3,6 раза**.
3	**Управление контекстом**	Пять подходов. ACON: на **26–54%** меньше токенов при точности 95%+.
4	**Цикл проверки**	Вычислительная (тесты) или inferential (LLM-судья, с задержкой).
5	**Права и безопасность**	Permissive: быстро, но рискованно. Restrictive: безопасно, но медленно.
6	**Набор инструментов**	Больше — часто хуже. Vercel убрал **80%** инструментов v0; Claude Code экономит **95%** контекста.
7	**Толщина harness'а**	Anthropic ставит на тонкий и убирает шаги, когда модель их усвоила.

HARNESS — ЭТО И ЕСТЬ ПРОДУКТ

Модель — товар, *harness — продукт*

1

Модели доступны всем

GPT, Claude и Gemini подключает через API любой разработчик.

2

Отличие уходит в обвязку

На одинаковых моделях продукты работают по-разному: в TerminalBench смена одного harness’а двигала агентов на 20+ позиций.

3

Это не коммодити

Контекст как дефицит, проверка, память без галлюцинаций, ставка на толщину — слой, где живёт сложный инжиниринг.

Клиенту продают не «доступ к ИИ», а выполненную задачу. И эту задачу делает harness.

Когда агент в следующий раз даст сбой — не вините модель. *Смотрите на harness.*

Спасибо за внимание! Вопросы?